ROOMBOOKING

Modèle de données & Dictionnaire de données

MCD · MLD · MPD · Dictionnaire de données

Document	Modèle de données
Version	1.0
Date	Février 2026
Auteur	Klaudia Juhasz
Organisation	MediaSchool IRIS Nice — BTS SIO SLAM
SGBD	MySQL 8.0 — Engine InnoDB — Charset utf8mb4

1. Introduction

Ce document décrit le modèle de données de l'application ***RoomBooking***. Il présente successivement le Modèle Conceptuel de Données (MCD), le Modèle Logique de Données (MLD), le Modèle Physique de Données (MPD) et le dictionnaire de données détaillant chaque attribut de chaque table.

La base de données comporte 10 tables :

Table	Type	Rôle
user	Entité	Compte utilisateur de base (email, mot de passe, nom)
administrator	Entité	Profil administrateur — extension de user
coordinator	Entité	Profil coordinateur — extension de user
student	Entité	Profil étudiant — extension de user, lié à une classe
classe	Entité	Groupe de formation (ex : BTS SIO 2)
coordinator_classe	Association	Lien ManyToMany coordinator ↔ classe
room	Entité	Salle réservable (nom, capacité)
equipment	Entité	Équipement disponible dans une salle
room_equipment	Association	Lien ManyToMany room ↔ equipment
reservation	Entité	Créneau de réservation d'une salle par un utilisateur

2. Modèle Conceptuel de Données (MCD)

Le MCD représente les entités métier, leurs attributs et les associations entre elles, indépendamment de toute implémentation technique. *(cf. Figure 1)*

2.1 Entités et associations

Association	Cardinalités	Description
faire	utilisateur (0,n) — reservation (1,1)	Un utilisateur fait 0 à n réservations. Une réservation est faite par exactement 1 utilisateur.
concerner	reservation (1,1) — room (0,n)	Une réservation concerne exactement 1 salle. Une salle est concernée par 0 à n réservations.

Association	Cardinalités	Description
posseder	room (0,n) — equipment (0,n)	Une salle possède 0 à n équipements. Un équipement peut être dans 0 à n salles.
appartient	student (1,1) — classe (1,n)	Un étudiant appartient à exactement 1 classe. Une classe contient 1 à n étudiants.
gestion_classe	coordinator (0,n) — classe (0,n)	Un coordinateur gère 0 à n classes. Une classe est gérée par 0 à n coordinateurs.
XT (Partition)	utilisateur → administrator, coordinator, student	Héritage exclusif : un utilisateur est soit administrateur, soit coordinateur, soit étudiant.

Info : La partition XT (exclusif-total) signifie que tout utilisateur appartient à exactement l'un des trois sous-types : *administrator, coordinator ou student.*

3. Modèle Logique de Données (MLD)

Le MLD traduit le MCD en tables relationnelles avec clés primaires et clés étrangères. Les clés primaires sont soulignées, les clés étrangères sont en italique. *(cf. Figure 2)*

user (#id, email, firstname, lastname, password)
administrator (#id, _user_id_)
coordinator (#id, _user_id_)
student (#id, _user_id_, _classe_id_)
classe (#id, name)
coordinator_classe (#_coordinator_id_, #_classe_id_)
room (#id, name, capacity)
equipment (#id, name)
room_equipment (#_room_id_, #_equipment_id_)
reservation (#id, reservation_start, reservation_end, status, _user_id_, _room_id_)

3.1 Relations entre tables

Table source	Table cible	Type de relation
administrator.user_id	user.id	OneToOne — 1 admin = 1 user unique
coordinator.user_id	user.id	OneToOne — 1 coordinator = 1 user unique
student.user_id	user.id	OneToOne — 1 student = 1 user unique
student.classe_id	classe.id	ManyToOne — 1 étudiant dans 1 classe (nullable)

Table source	Table cible	Type de relation
coordinator_classe.coordinator_id	coordinator.id	ManyToMany via table de liaison
coordinator_classe.classe_id	classe.id	ManyToMany via table de liaison
room_equipment.room_id	room.id	ManyToMany via table de liaison
room_equipment.equipment_id	equipment.id	ManyToMany via table de liaison
reservation.user_id	user.id	ManyToOne — 1 réservation = 1 bénéficiaire
reservation.room_id	room.id	ManyToOne — 1 réservation = 1 salle

4. Modèle Physique de Données (MPD)

Le MPD correspond à l'implémentation réelle dans MySQL 8.0. Toutes les tables utilisent le moteur InnoDB avec l'encodage utf8mb4 pour la gestion complète des caractères Unicode.

```
-- Table : user
CREATE TABLE `user` (
  `id`        INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `email`     VARCHAR(255) NOT NULL UNIQUE,
  `firstname` VARCHAR(100) NOT NULL,
  `lastname`  VARCHAR(100) NOT NULL,
  `password`  VARCHAR(255) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table : administrator
CREATE TABLE `administrator` (
  `id`      INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `user_id` INT NOT NULL UNIQUE,
  FOREIGN KEY (`user_id`) REFERENCES `user`(`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table : coordinator
CREATE TABLE `coordinator` (
  `id`      INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `user_id` INT NOT NULL UNIQUE,
  FOREIGN KEY (`user_id`) REFERENCES `user`(`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table : student
CREATE TABLE `student` (
  `id`        INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `user_id`   INT NOT NULL UNIQUE,
  `classe_id` INT DEFAULT NULL,
  FOREIGN KEY (`user_id`)   REFERENCES `user`(`id`),
  FOREIGN KEY (`classe_id`) REFERENCES `classe`(`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```sql
-- Table : classe
CREATE TABLE `classe` (
  `id`   INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(150) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table de liaison : coordinator_classe
CREATE TABLE `coordinator_classe` (
  `coordinator_id` INT NOT NULL,
  `classe_id`      INT NOT NULL,
  PRIMARY KEY (`coordinator_id`, `classe_id`),
  FOREIGN KEY (`coordinator_id`) REFERENCES `coordinator`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`classe_id`)      REFERENCES `classe`(`id`)      ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table : room
CREATE TABLE `room` (
  `id`       INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `name`     VARCHAR(150) NOT NULL,
  `capacity` INT NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table : equipment
CREATE TABLE `equipment` (
  `id`   INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(150) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table de liaison : room_equipment
CREATE TABLE `room_equipment` (
  `room_id`      INT NOT NULL,
  `equipment_id` INT NOT NULL,
  PRIMARY KEY (`room_id`, `equipment_id`),
  FOREIGN KEY (`room_id`)      REFERENCES `room`(`id`)      ON DELETE CASCADE,
  FOREIGN KEY (`equipment_id`) REFERENCES `equipment`(`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table : reservation
CREATE TABLE `reservation` (
  `id`               INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `reservation_start` DATETIME NOT NULL,
  `reservation_end`   DATETIME NOT NULL,
  `status`           VARCHAR(20) NOT NULL DEFAULT 'active',
  `user_id`          INT NOT NULL,
  `room_id`          INT NOT NULL,
  FOREIGN KEY (`user_id`) REFERENCES `user`(`id`),
  FOREIGN KEY (`room_id`) REFERENCES `room`(`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5. Dictionnaire de données

Le dictionnaire de données précise pour chaque attribut : son nom exact en base, son type SQL, ses contraintes et sa description fonctionnelle.

5.1 Table : user

Compte utilisateur de base commun à tous les profils.

TABLE : user			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique auto-incrémenté
email	VARCHAR(255)	NOT NULL, UNIQUE	Adresse email — identifiant de connexion unique dans toute l'application
firstname	VARCHAR(100)	NOT NULL	Prénom de l'utilisateur
lastname	VARCHAR(100)	NOT NULL	Nom de famille de l'utilisateur
password	VARCHAR(255)	NOT NULL	Mot de passe hashé en bcrypt (jamais stocké en clair)

5.2 Table : administrator

Profil administrateur — extension de user via OneToOne. L'existence d'une ligne ici confère le rôle ROLE_ADMIN.

TABLE : administrator			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de l'administrateur
user_id	INT	FK → user.id, NOT NULL, UNIQUE	Référence vers le compte user associé (relation OneToOne)

5.3 Table : coordinator

Profil coordinateur (intervenant) — extension de user via OneToOne. Confère le rôle ROLE_COORDINATOR.

TABLE : coordinator			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique du coordinateur
user_id	INT	FK → user.id, NOT NULL, UNIQUE	Référence vers le compte user associé (relation OneToOne)

5.4 Table : student

Profil étudiant — extension de user via OneToOne. Un étudiant peut être rattaché à une classe ou en être détaché (nullable).

TABLE : student			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de l'étudiant
user_id	INT	FK → user.id, NOT NULL, UNIQUE	Référence vers le compte user associé (relation OneToOne)
classe_id	INT	FK → classe.id, DEFAULT NULL	Classe d'appartenance — nullable : un étudiant peut être retiré d'une classe sans supprimer son compte

5.5 Table : classe

Groupe de formation (promotion). Contient des étudiants (OneToMany) et est gérée par des coordinateurs (ManyToMany).

TABLE : classe			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de la classe
name	VARCHAR(150)	NOT NULL	Nom de la classe (ex : BTS SIO 2, LP SLAM...)

5.6 Table : coordinator_classe

Table de liaison ManyToMany entre coordinator et classe. Un coordinateur peut gérer plusieurs classes et une classe peut avoir plusieurs coordinateurs.

TABLE : coordinator_classe			
Colonne	**Type SQL**	**Contraintes**	**Description**
coordinator_id	INT	PK composite, FK → coordinator.id, ON DELETE CASCADE	Identifiant du coordinateur
classe_id	INT	PK composite, FK → classe.id, ON DELETE CASCADE	Identifiant de la classe

Info : La clé primaire composite (coordinator_id, classe_id) garantit l'unicité du lien — un coordinateur ne peut être associé qu'une seule fois à la même classe.

5.7 Table : room

Salle réservable. Peut posséder plusieurs équipements (ManyToMany) et avoir plusieurs réservations (OneToMany).

TABLE : room			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de la salle
name	VARCHAR(150)	NOT NULL	Nom de la salle (ex : Salle A101, Lab Informatique 2...)
capacity	INT	NOT NULL	Nombre maximum de personnes pouvant occuper la salle

5.8 Table : equipment

Équipement disponible dans une ou plusieurs salles. Relation ManyToMany avec room via room_equipment.

TABLE : equipment			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de l'équipement
name	VARCHAR(150)	NOT NULL	Nom de l'équipement (ex : Vidéoprojecteur, Tableau blanc, PC fixe...)

5.9 Table : room_equipment

Table de liaison ManyToMany entre room et equipment. Un équipement peut exister dans plusieurs salles et une salle peut avoir plusieurs équipements.

TABLE : room_equipment			
Colonne	**Type SQL**	**Contraintes**	**Description**
room_id	INT	PK composite, FK → room.id, ON DELETE CASCADE	Identifiant de la salle
equipment_id	INT	PK composite, FK → equipment.id, ON DELETE CASCADE	Identifiant de l'équipement

⚠ **Attention :** ON DELETE CASCADE : si une salle ou un équipement est supprimé, les lignes correspondantes dans room_equipment sont automatiquement supprimées.

5.10 Table : reservation

Créneau de réservation d'une salle par un utilisateur. Les annulations sont gérées en soft delete via le champ status.

TABLE : reservation			
Colonne	**Type SQL**	**Contraintes**	**Description**
id	INT	PK,	Identifiant unique de la

		AUTO_INCREMENT, NOT NULL	réservation
reservation_start	DATETIME	NOT NULL	Date et heure de début du créneau (format : YYYY-MM-DD HH:MM:SS)
reservation_end	DATETIME	NOT NULL	Date et heure de fin du créneau (doit être > reservation_start)
status	VARCHAR(20)	NOT NULL, DEFAULT 'active'	Statut de la réservation : 'reserved' (en cours / à venir) ou 'canceled' (annulée — soft delete)
user_id	INT	FK → user.id, NOT NULL	Bénéficiaire de la réservation (peut être différent de l'auteur pour les coordinateurs)
room_id	INT	FK → room.id, NOT NULL	Salle réservée

Bon à savoir : Le soft delete sur status = 'canceled' permet de conserver l'historique complet des réservations annulées sans les supprimer physiquement de la base.

6. Règles d'intégrité

Règle	Description
Email unique	La colonne user.email est UNIQUE — impossible de créer deux comptes avec le même email
OneToOne user ↔ profil	user_id est UNIQUE dans administrator, coordinator et student — un user ne peut avoir qu'un seul profil
Suppression classe	Une classe ne peut être supprimée en application que si elle n'a plus d'étudiant (contrainte applicative Symfony)
Suppression salle	Une salle ne peut être supprimée en application que si elle n'a plus de réservation
student.classe_id nullable	Un étudiant peut être retiré d'une classe sans que son compte soit supprimé (classe_id = NULL)
Cascade coordinator_classe	ON DELETE CASCADE — si un coordinateur ou une classe est supprimé, les liens sont automatiquement retirés

Règle	Description
Cascade room_equipment	ON DELETE CASCADE — si une salle ou un équipement est supprimé, les liens sont automatiquement retirés
Conflit de réservation	Contrôle applicatif dans RoomRepository::isAvailable() — vérifie les chevauchements avant toute insertion
Soft delete réservation	Les annulations ne suppriment pas la ligne — elles passent status à 'canceled' pour conserver l'historique

Annexes

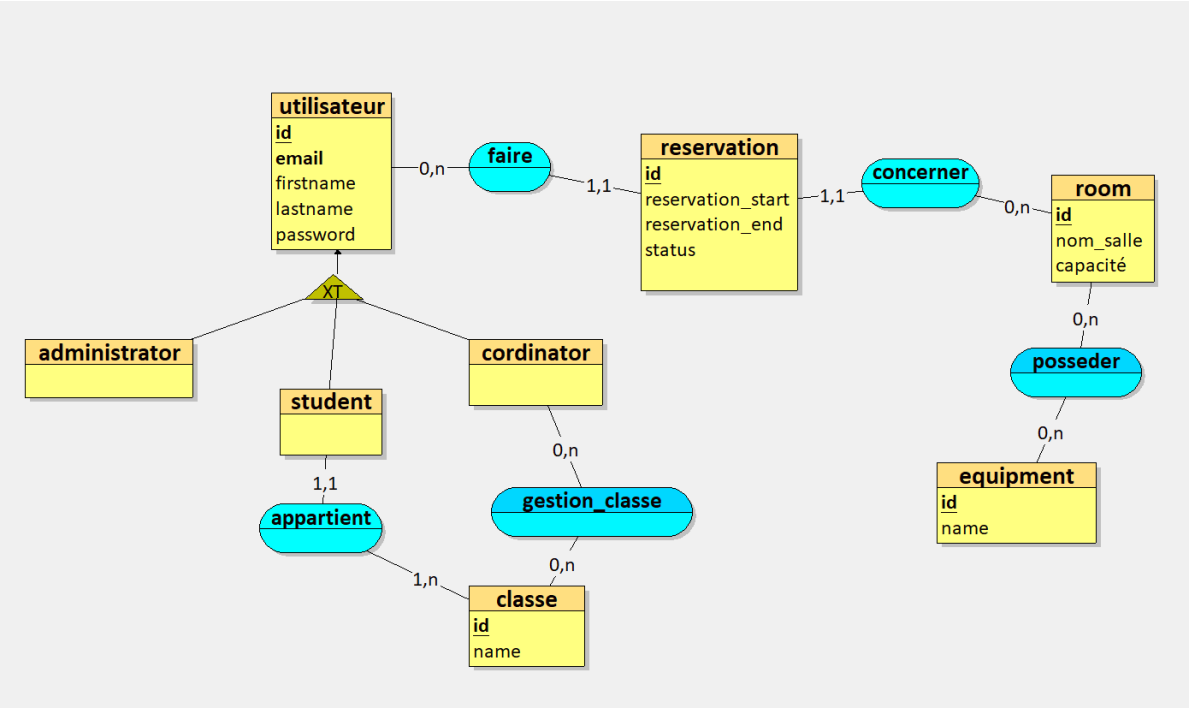

Figure 1: Modèle Conceptuel de Données

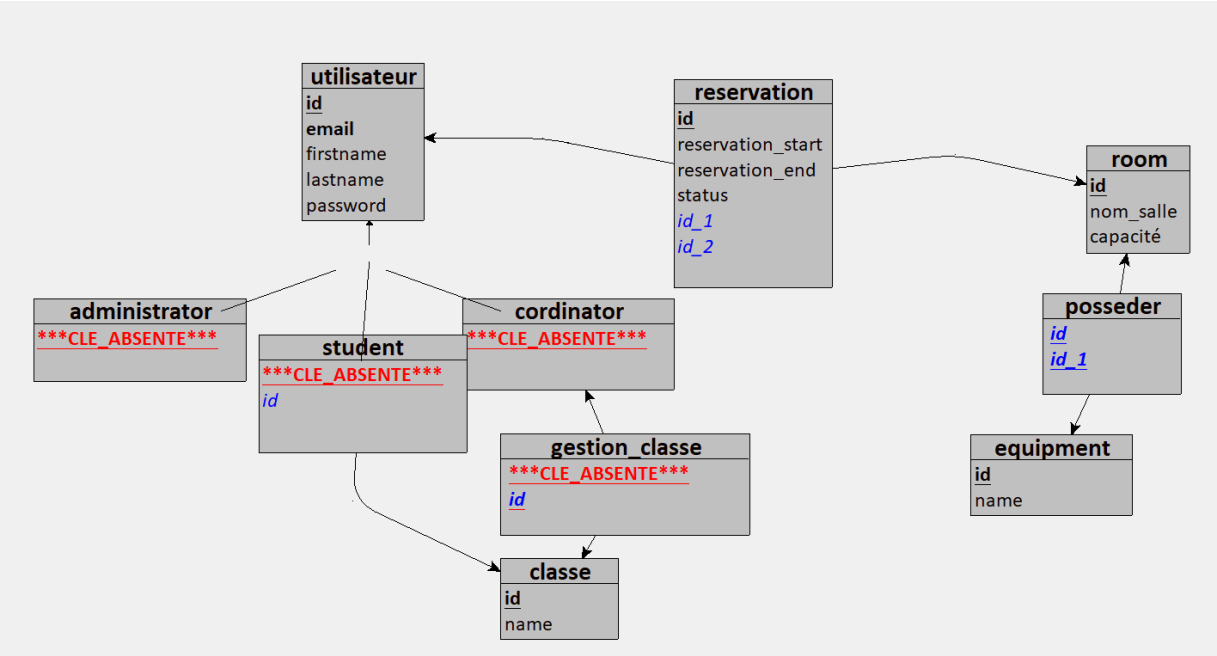

Figure 2: Modèle Logique de Données

MediaSchool IRIS Nice — BTS SIO SLAM — Février 2026